



Amazon Cognito

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Identity Providers (IdP)

Identity Provider (IdP) a system entity that creates, maintains, and manages identity information for principals and also provides authentication services to applications within a **federation** or distributed network. A trusted provider of your user identity that lets you authenticate to access other services.

Identity Providers could be: **Facebook, Amazon, Google, Twitter, Github, LinkedIn**

Federated identity is a method of linking a user's identity across multiple separate identity management systems



OpenID

open standard and decentralized authentication protocol. Eg be able to login into a different social media platform using a Google or Facebook account

OpenID is about providing who are you



OAuth2.0

industry-standard protocol for authorization OAuth doesn't share password data but instead uses authorization tokens to prove an identity between consumers and service providers.

Oauth is about granting access to functionality



SAML

Security Assertion Markup Language is an open standard for exchanging authentication and authorization between an identity provider and a service provider.

An important use case for SAML is Single-Sign-On via web browser.

Authentication vs Authorization

Authentication **validates an identity**

Is this person Andrew?



Open ID Connect (OCID) is
used for Authentication

Authorization **grants access a resource.**

Does Andrew have access to this web-app?



OAuth2 is used
for Authorization

The OAuth 2.0 Authorization Framework

The OAuth2 Authorization Framework is an **online technical documentation** containing the proposed **implementation standards** for OAuth2

The document is presented as a **Request for Comments (RFC)**

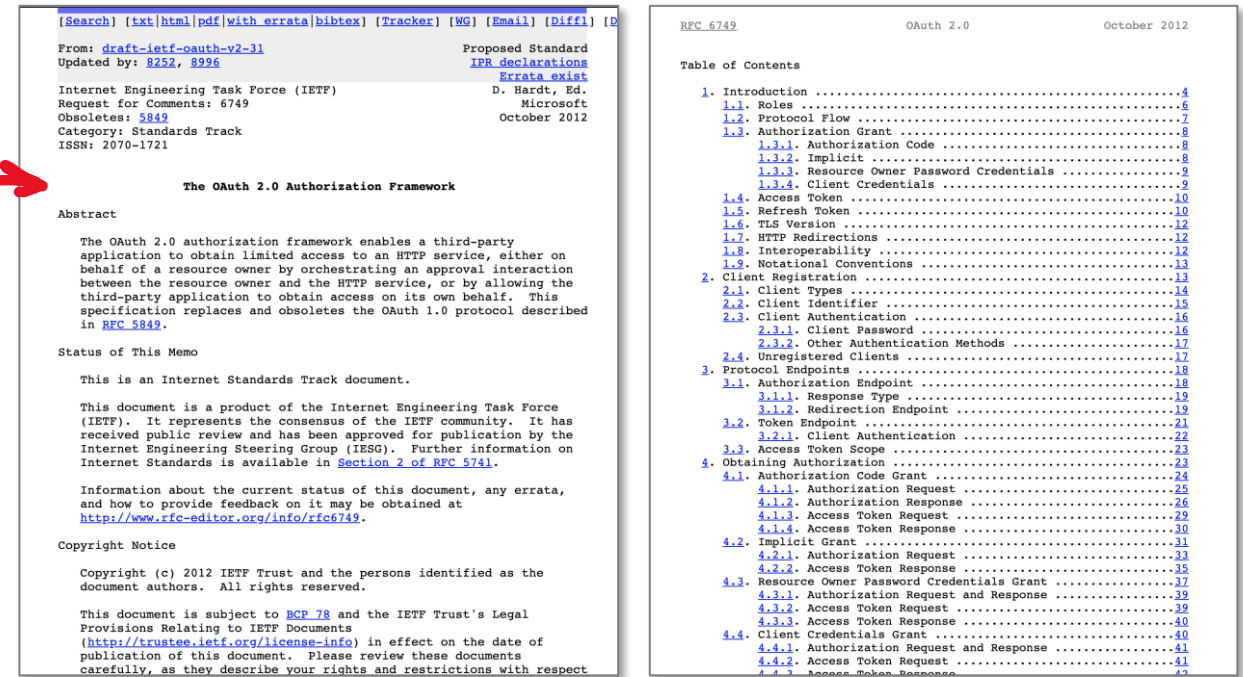
RFCs are authored by individuals or groups of engineers and computer scientists.

RFCs are in the form of a memorandum describing methods, behaviors, research, or innovations applicable to the working of the Internet and Internet-connected systems.

RFCs are submitted either for peer review or to convey new concepts and information

RFCs are informational or experimental in nature and **are not standards**

The IETF adopts some of the proposals published as RFCs as Internet Standards.



The framework is freely viewable at:

<https://datatracker.ietf.org/doc/html/rfc6749>

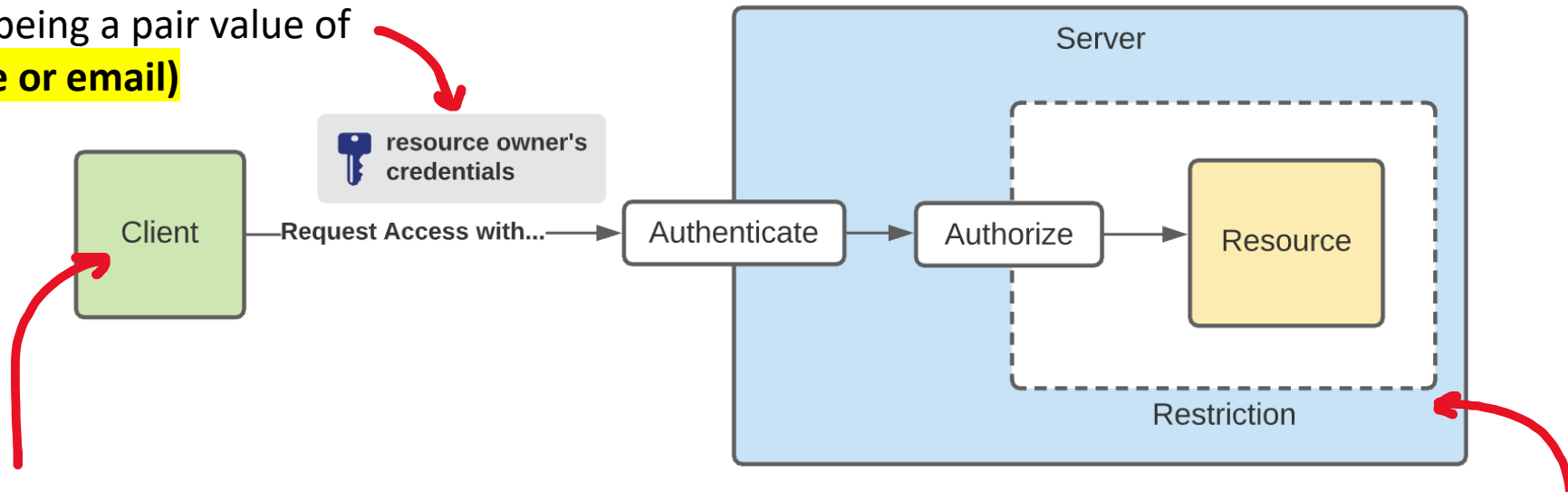
Internet Engineering Task Force (IETF) is the principal technical development and standards-setting bodies for the Internet.

Traditional Client-Server Authentication Model

The **client** requests an **access-restricted resource** (protected resource) on the **server** by authenticating with the server using the resource owner's credentials.

The credentials being a pair value of

- **ID (username or email)**
- **Password**



“The client” can be a **client-side web-app**
eg. Single Page Application (SPA) built as a React app

“The server” can be the **server-side web-app**
eg. Ruby on Rails that serves an API

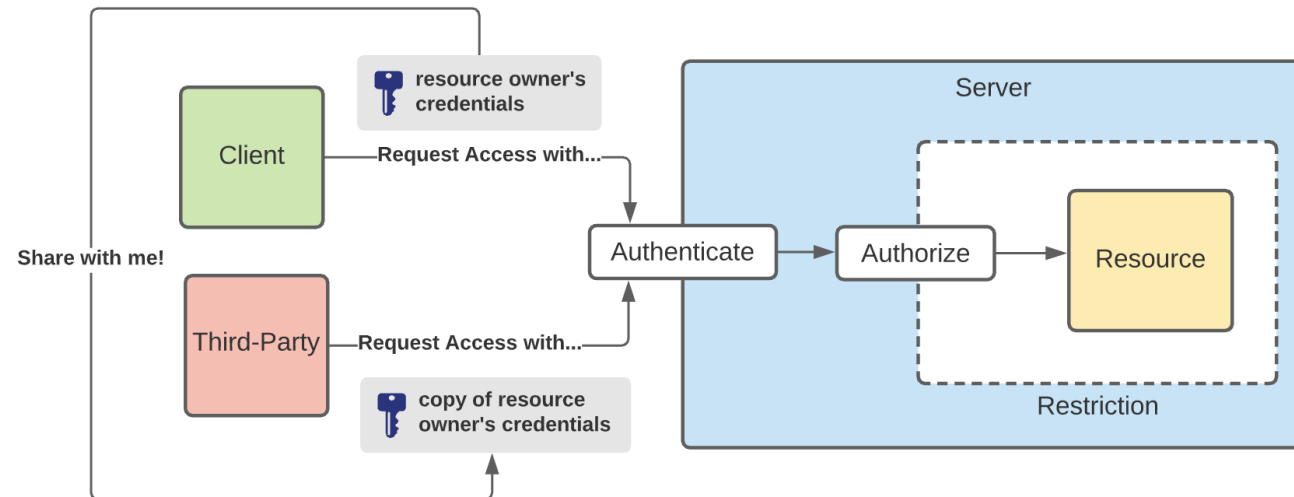
When the client and server is owned and managed by a single party it is easy to ensure the end-to-end security for our identity.

But what happens when we need to authenticate via a third party?

Flaw of Traditional Client-Server Authentication Model

A **third party** could be a partnered business who wants to access API endpoints to your backend server to fetch data for its own customers

A simple (and flawed) solution to allowing them to access your server's resources is by **sharing with them a copy of the credentials** used to authenticate to the server



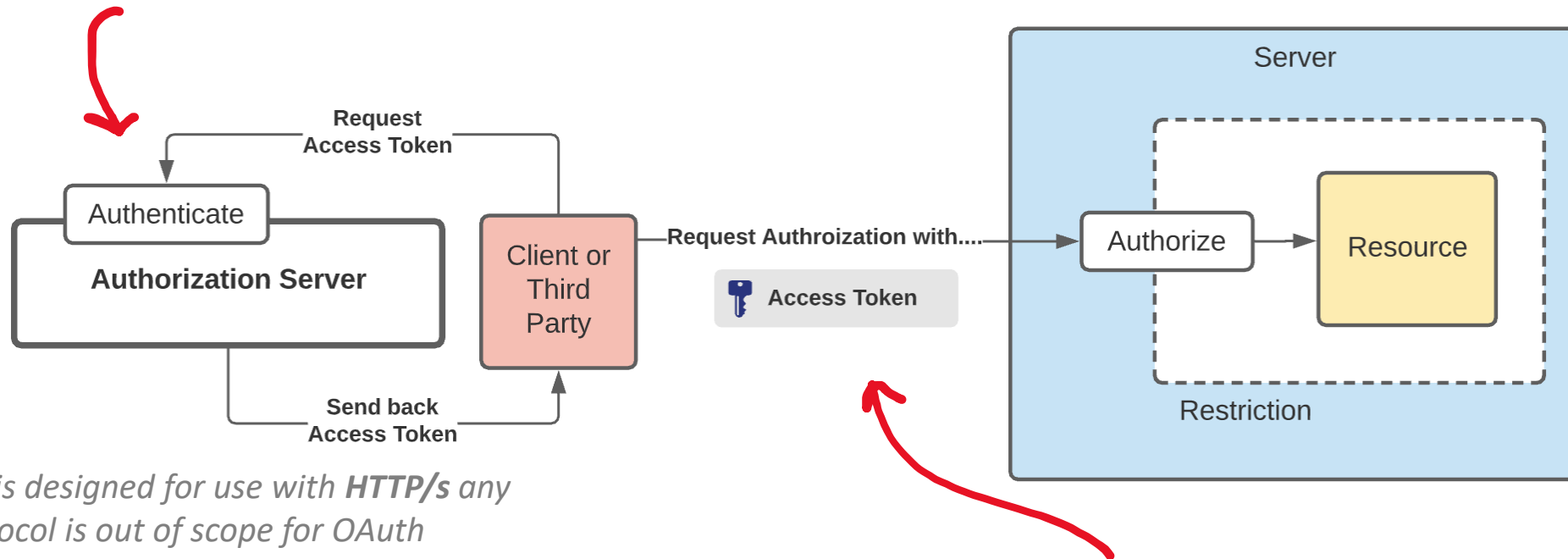
Here are the issues:

- Third-party app are required to store the resource owner's credentials for future use **in plain-text**
- Servers will be required to support password authentication, despite the security weaknesses inherent in passwords.
- Third-party app gain overly broad access to the resource owner's protected resource
 - leaving resource owners **without any ability to restrict duration or access to a limited subset of resources.**
- Resource owners **cannot revoke access to an individual third party without revoking access to all third parties**
 - must do so by changing the third party's password.
- Compromise of any third-party apps results in compromise of the end-user's password and all of the data protected by that password

OAuth2 Authentication Model

OAuth addresses these issues by introducing an **authorization layer** and separating the role of the client from that of the resource owner

A third party authenticates with a **trusted server** (authorization server) and gets back an access token



*OAuth2.0 is designed for use with **HTTP/s** any other protocol is out of scope for OAuth*

The third-party using that session data can request an **access token** from an authorization server
The access token a string denoting a specific **scope, lifetime**, and **other access attributes**

Access tokens are issued to third-party clients by an authorization server **with the approval of the resource owner**.

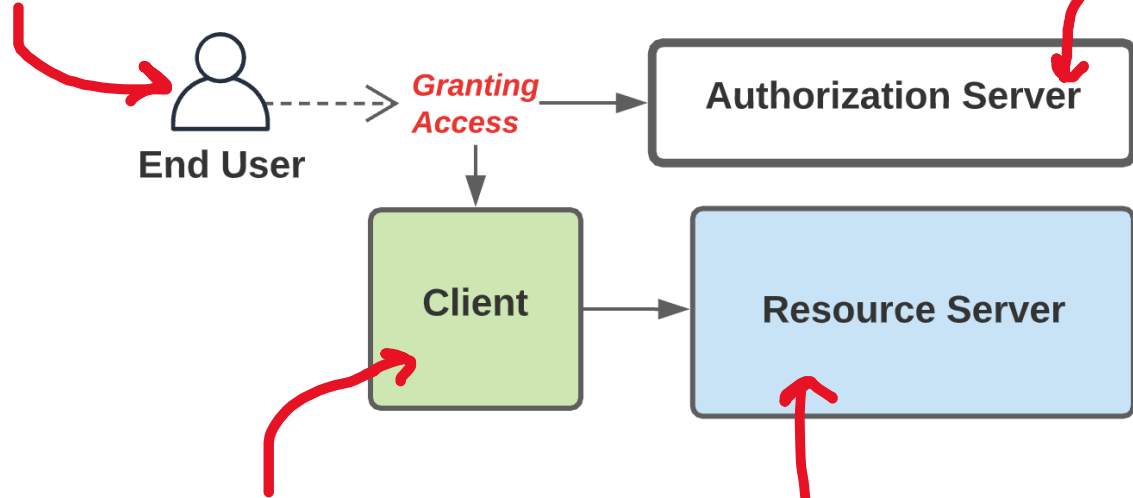
OAuth2 Roles

A role is the entity that has a specific responsibility in the OAuth framework

OAuth has 4 roles

resource owner

An entity capable of **granting access** to a protected resource.
When the resource owner is a person, it is referred to as an end-user.



client

An app **making protected resource requests on behalf of the resource owner and with its authorization**.
The term "client" does not imply implementation characteristics eg. server, desktop, or other device.

authorization server

The **server issuing access tokens** to the client after successfully authenticating the resource owner and obtaining authorization.

The authorization server may be the same server as the resource server or a separate entity.

A single authorization server may issue access tokens accepted by multiple resource servers.

resource server

The server hosting the protected resources, capable of **accepting and responding to protected resource requests** using access tokens.

OAuth2 Protocol Flow

The protocol flow **describes the interactions** between the 4 OAuth Roles



The client requests authorization from the resource owner.

The request can be made directly (as shown), or indirectly via the authorization server

OAuth2 Protocol Flow

The protocol flow **describes the interactions** between the 4 OAuth Roles



The client receives an authorization grant, which is a credential representing the resource owner's authorization, expressed using **one of four grant types** defined in this specification or using an **extension grant type**. The authorization grant type depends on the method used by the client to request authorization and the types supported by the authorization server.

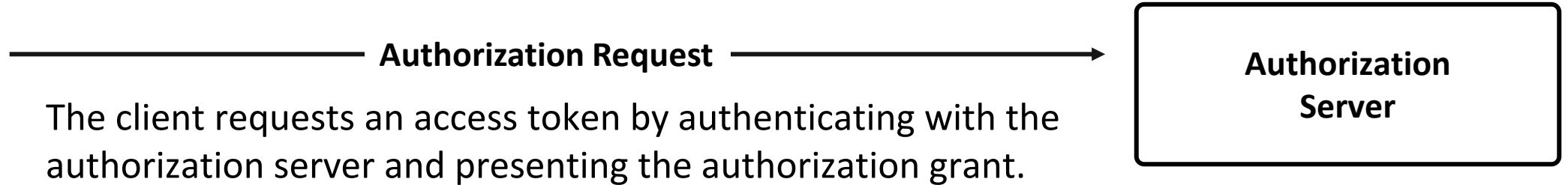
Authentication Grant can be of the following types:

- **authorization code**
- Implicit
- Resource Owner Password Credentials
- Client Credentials

The preferred method for the client to obtain an authorization grant from the resource owner is to use the authorization server as an **intermediary** eg. Authorization Code Grant

OAuth2 Protocol Flow

The protocol flow **describes the interactions** between the 4 OAuth Roles



OAuth2 Protocol Flow

The protocol flow **describes the interactions** between the 4 OAuth Roles



The authorization server authenticates the client and validates the authorization grant, and if valid, issues an access token

OAuth2 Protocol Flow

The protocol flow **describes the interactions** between the 4 OAuth Roles

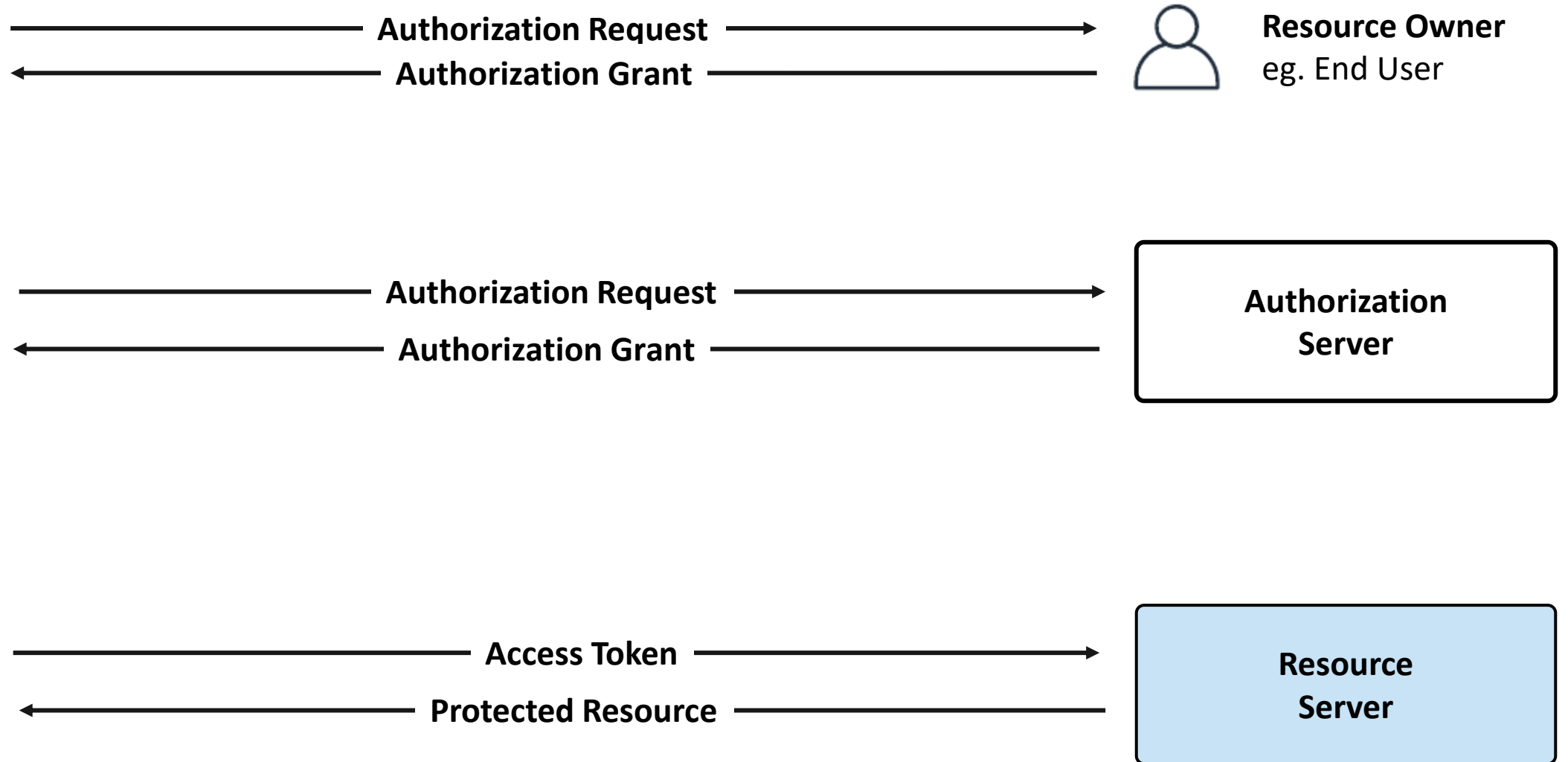


The client requests the protected resource from the resource server and authenticates by presenting the access token.



OAuth2 Protocol Flow

The protocol flow **describes the interactions** between the 4 Oauth Roles



The resource server validates the access token, and if valid, serves the request.

JSON Web Token (JWT)

JSON Web Token (JWT) is a compact, URL-safe means of representing claims to be transferred between two parties. The information can be verified and trusted because it is digitally signed. JWTs can be used in authentication systems to identify active sessions. JWTs can be stored in LocalStorage in the browser.

A JSON Web Token (JWT) is composed of the following:

- Header
 - the type of the token eg. JWT
 - signing algorithm eg. HMAC
- Payload
 - Contains the claims
 - Claims are statements about an entity
- Signature
 - Verifies the sender
 - Ensures Data Integrity
 - Prevents Forgery of Data

Header

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "admin": true,  
  "iat": 1516239022  
}
```

Signature

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdW  
IiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9l  
IiwiaWF0Ij01NjM0NTY3ODkwIiwiaWF0Ij01NjM0NTY3ODkw  
dPxLDWy7C-7rlu9UHw1e12RkI8yG5Aag7GkLHR8EK9k
```

JSON Web Token (JWT)

Example of a JWT encoded URL. Notice the pattern **<header>.<payload>.<signature>**

Encoded

PASTE A TOKEN HERE

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0Ij0iNjIyOTYyMn0.reGQzG30KdoIMWLDK0Z4TICJit3EW69cQE72E2CfzRE

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true,
  "iat": 1516239022
}
```

VERIFY SIGNATURE

HMACSHA256(
 base64UrlEncode(header) + "." +
 base64UrlEncode(payload),

) ☐ secret base64 encoded

Signature Verified

SHARE JWT

JSON Web Token (JWT)

Example of **decoding a JWT** server side using Ruby to get to the claims data.

```
require 'jwt'

# Your JWT token
token =
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0eSkiOnRydWUsImVudCI6MTUxNjIzOTAyMn0.dPxLDWy7C-7rLu9UHwle12RkI8yG5Aag7GkLHR8EK9k'

# The secret key used to sign the JWT
secret = 'your-256-bit-secret'

begin
  # Decode the token
  decoded_token = JWT.decode token, secret, true, { algorithm: 'HS256' }

  # Output the decoded token
  puts "Decoded JWT: #{decoded_token}"
rescue JWT::DecodeError => e
  puts "Invalid token: #{e.message}"
end
```

JSON Web Key Sets (JWKS)

JSON Web Key Set (JWKS) is a standard for **publishing and distributing cryptographic keys** in a JSON format.

JWKS facilitate the sharing of public keys that can be used to **verify the signatures of JSON Web Tokens (JWTs)** and encrypt or decrypt messages.

Structure of a JWKS



```
{
  "keys": [
    {
      "kty": "RSA",
      "use": "sig",
      "kid": "12345",
      "alg": "RS256",
      "n": "0vx7agoebGcQSuuP..",
      "e": "AQAB"
    },
    // more keys
  ]
}
```

- kty (Key Type): specific cryptographic algorithm family used with the key
- use (Public Key Use): how the key was meant to be used; for signatures (sig) or encryption (enc).
- alg (Algorithm): Identifies the algorithm intended for use with the key,
- kid (Key ID): A unique identifier for the key, which is used to match a specific key to a JWT that has a kid claim in its header.
- n and e (RSA Public Key parameters): When the key type is RSA, these parameters represent the modulus and exponent, respectively.
- x and y (Elliptic Curve parameters): For keys of type EC, these represent the coordinates on the curve.

JSON Web Key Sets (JWKS)

Example of **verifying**
with a JWKS



```
require 'jwt'
require 'net/http'
require 'json'

# The URL of the JWKS
jwks_url = 'https://example.com/.well-known/jwks.json'

# The JWT you received
token = 'your.jwt.token.here'

# Function to fetch the JWKS
def fetch_jwks(url)
  uri = URI(url)
  response = Net::HTTP.get(uri)
  JSON.parse(response)['keys']
end

# Function to find the appropriate key
def find_key(jwks, kid)
  jwks.find { |key| key['kid'] == kid }
end

begin
  # Decode the JWT header without verification to extract the kid
  header = JWT.decode(token, nil, false).first['header']
  kid = header['kid']

  # Fetch JWKS and select the appropriate key
  jwks = fetch_jwks(jwks_url)
  jwk_data = find_key(jwks, kid)

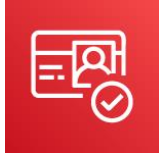
  # If no key is found with the provided kid, raise an error
  raise "Unable to find appropriate key" unless jwk_data

  # Create a public key instance from the JWK data
  key = OpenSSL::PKey::RSA.new(
    JWT::JWK.import(jwk_data).keypair
  )

  # Decode and verify the JWT using the public key
  decoded_token = JWT.decode token, key, true, { algorithm: 'RS256' }

  # Out
  puts "Decoded JWT: #{decoded_token}"
rescue JWT::DecodeError => e
  puts "Invalid token: #{e.message}"
rescue => e
  puts "Error: #{e.message}"
end
```

Introduction to Amazon Cognito



Amazon Cognito is a **customer identity and access management** (CIAM) system. It provides authentication, authorization, and user management for your web and mobile apps. It also provides authentication to AWS Services.

Cognito User Pools

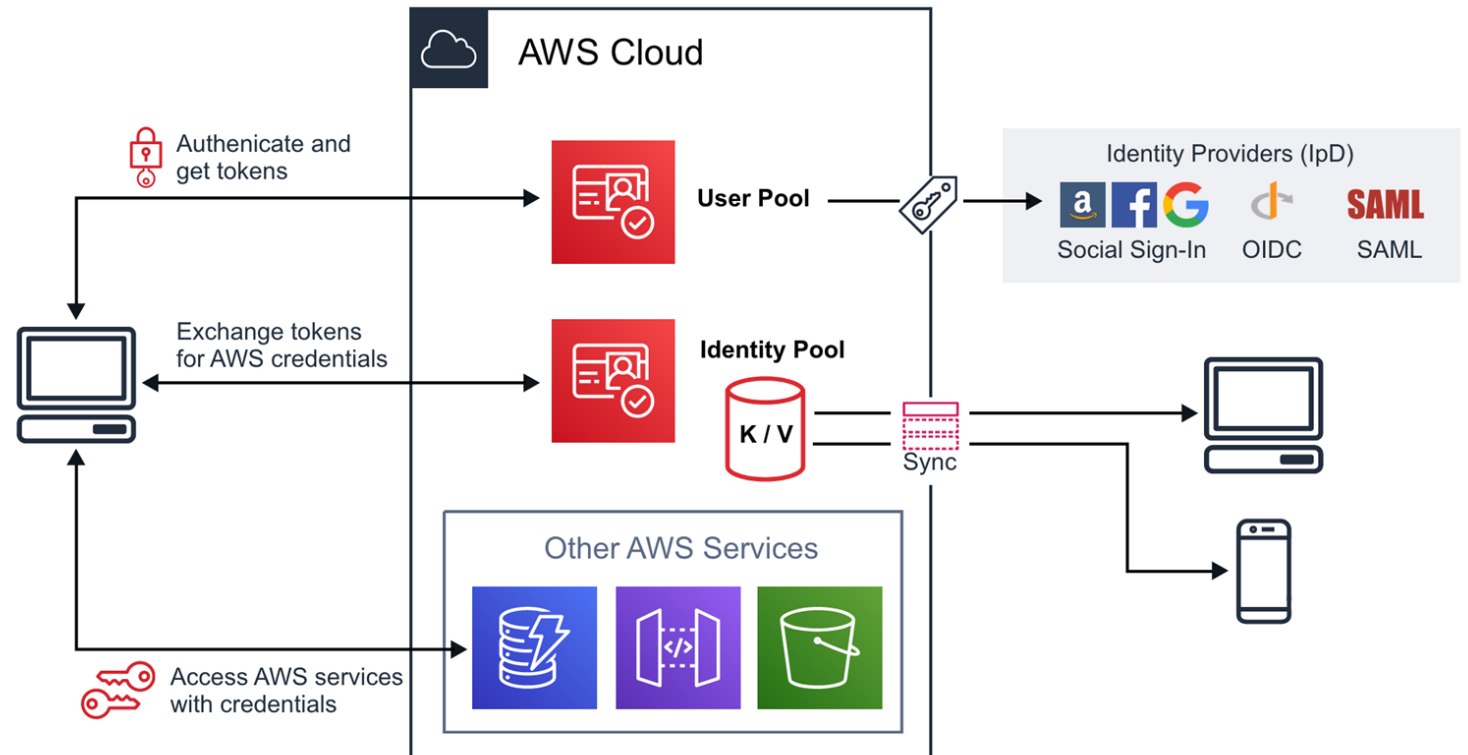
User directory with authentication to IdP to grant access to your app.

Cognito Identity Pools

Provide temporary credentials for users to access AWS Services

Cognito Sync

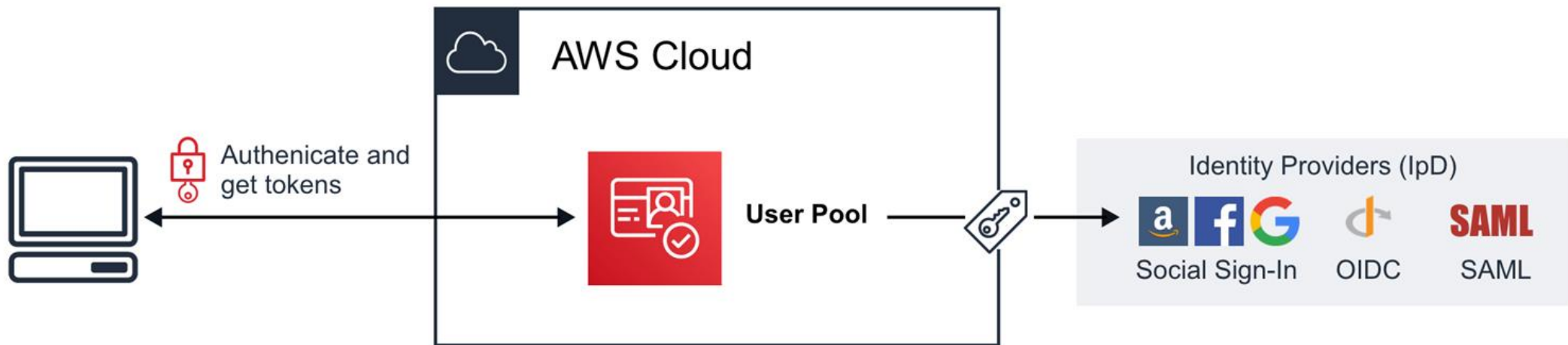
Syncs user data and preferences across all devices



Cognito User Pools

User Pools are **user directories** used to manage the actions for web and mobile apps:

- eg. Sign-up, Sign-in, Account recovery. Account confirmation
- Allows users to sign in directly to the User Pool or use Web Identity Federation.
- Uses AWS Cognito as the identity broker between AWS and the identity provider.
- If Successful, user authentication generates a JSON Web Token (JWTs).
- User Pools can be thought of as the account used to access the system (ie email address and password)



Cognito charges based on Monthly Active Users (MAUs)

Cognito User Pools – App Client

App clients are configurations within a user pool that interacts with one mobile or web application that authenticates with Amazon Cognito.

When you make API calls to Cognito you have to supply a App Client ID

For each app client in your user pool, you can sign in your users with any combination of one or more flows,

ExplicitAuthFlows:

The authentication flows that you want your user pool client to support.

AllowedOAuthFlows

The OAuth grant types that you want your app client to generate.

```
MyCognitoUserPoolClient:
  Type: AWS::Cognito::UserPoolClient
  Properties:
    ClientName: MyAppClient
    UserPoolId: !Ref MyCognitoUserPool
    GenerateSecret: false
    ExplicitAuthFlows:
      - ALLOW_USER_SRP_AUTH
      - ALLOW_REFRESH_TOKEN_AUTH
    AllowedOAuthFlows:
      - code
      - implicit
    AllowedOAuthScopes:
      - phone
      - email
      - openid
      - profile
    CallbackURLs:
      - https://www.example.com/callback
    LogoutURLs:
      - https://www.example.com/logout
```

Cognito User Pools – App Client

ExplicitAuthFlows

- **ALLOW_ADMIN_USER_PASSWORD_AUTH:**
 - Enable admin based user password authentication flow ADMIN_USER_PASSWORD_AUTH.
 - This setting replaces the ADMIN_NO_SRP_AUTH setting.
 - With this authentication flow, your app passes a user name and password to Amazon Cognito in the request, instead of using the Secure Remote Password (SRP) protocol to securely transmit the password.
- **ALLOW_CUSTOM_AUTH:**
 - Enable Lambda trigger based authentication.
- **ALLOW_USER_PASSWORD_AUTH:**
 - Enable user password-based authentication. In this flow, Amazon Cognito receives the password in the request instead of using the SRP protocol to verify passwords.
- **ALLOW_USER_SRP_AUTH:**
 - Enable SRP-based authentication.
- **ALLOW_REFRESH_TOKEN_AUTH:**
 - Enable authflow to refresh tokens.

Cognito User Pools – App Client

AllowedOAuthFlows

- code
 - Use a code grant flow, which provides an authorization code as the response.
 - This code can be exchanged for access tokens with the /oauth2/token endpoint.
- implicit
 - Issue the access token (and, optionally, ID token, based on scopes) directly to your user.
- client_credentials
 - Issue the access token from the /oauth2/token endpoint directly to a non-person user using a combination of the client ID and client secret.
 - To create an app client that generates client credentials grants, you must add client_credentials as the only allowed OAuth flow.

Cognito User Pools – Password Policy

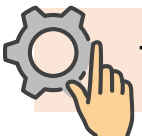
When you create a Cognito User Pool you can set a **Password Policy**

```
Resources:
  MyCognitoUserPool:
    Type: AWS::Cognito::UserPool
    Properties:
      UserPoolName: MyUserPool
      Policies:
        PasswordPolicy:
          MinimumLength: 8
          RequireUppercase: true
          RequireLowercase: true
          RequireNumbers: true
          RequireSymbols: true
```

TemporaryPasswordValidityDays

The number of days a temporary password is valid in the password policy.

If the user doesn't sign in during this time, an administrator must reset their password. Defaults to 7.



There is no password expiry like IAM Password Policy so you need to make manual effort to force password rotation

Cognito User Pools – Attributes

Attributes are pieces of information that help you identify individual users, such as name, email address, and phone number.

Amazon Cognito assigns all users a set of standard attributes based on **the OpenID Connect specification**

- address
- birthdate
- email
- family_name
- gender
- given_name
- locale
- middle_name
- name
- nickname
- phone_number
- picture
- preferred_username
- profile
- **Sub** —The unique identifier. Its required and set by Cognito
- updated_at
- website
- zoneinfo

- You can add up to 50 custom attributes to your user pool
- You define your **attribute schema** at the time of creation



```
Resources:
  MyCognitoUserPool:
    Type: AWS::Cognito::UserPool
    Properties:
      UserPoolName: MyUserPool
      Schema:
        - Name: email
          AttributeDataType: String
          Required: true
          Mutable: false
        - Name: phone_number
          AttributeDataType: String
          Required: true
          Mutable: true
        - Name: custom:membershipID
          AttributeDataType: String
          Required: false
          Mutable: true
```

Cognito User Pools – Advanced Security

Advanced Security are **additional security options** you can enable after the creation of a User Pool.

Compromised credentials

Using public data can determine if you a user account password is compromised.

Follow up action:

- Allow sign-in
- Block sign-in

Adaptive authentication

review location and device information from your users' sign-in requests and apply an automatic response to secure the user accounts in your user pool against suspicious activity.

Follow up action:

- Block sign-in
- Required MFA

Access token customization

configure your user pool to accept responses to a version 2 Lambda trigger event.

create flexible authorization outcomes when your users authenticate

Advanced Security will result in additional charges based on MAUs

Cognito User Pools – Email Settings

Amazon Cognito generally delivers emails to users for various follows eg. Signup, Reset

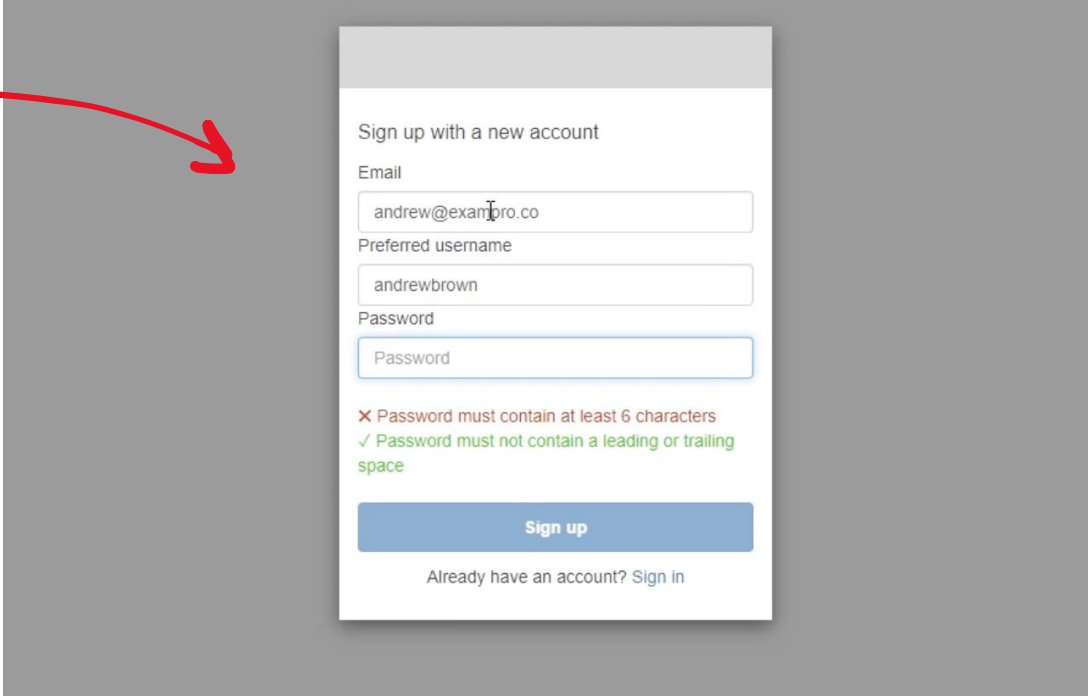
There are two email delivery options:

- **Built-in** email delivery by Cognito
 - default email address, no-reply@verificationemail.com
 - 50 Email messages sent daily per AWS account
 - Limited email customization
- **SES Configuration**
 - Request amount of emails you need per month to AWS Service Limits eg 50K+ email
 - Advanced Analytics
 - Create fully customized email templates

Cognito User Pools – Hosted UI

Cognito User Pools has **Hosted UI** in which AWS **hosts all the authentication and authorization pages**

- You can use a custom domain to a friendly URL.
- Your app will direct to the Hosted UI and redirect back to your app



Sign up with a new account

Email

Preferred username

Password

✗ Password must contain at least 6 characters
✓ Password must not contain a leading or trailing space

[Sign up](#)

Already have an account? [Sign in](#)

`https://<your domain>/oauth2/authorize?response_type=code&client_id=<your app client id>&redirect_uri=<your callback url>`

Without a custom domain your page will appear at hosted shared domain

- eg. usergroupname.auth.ca-central1.amazoncognito.com

Cognito User Pools – Lambda Triggers

Amazon Cognito works with AWS Lambda functions to modify the authentication behavior of your user pool.

Lambda Triggers

- Pre sign-up
- **Post confirmation**
- Pre authentication
- Post authentication
- Challenge Lambda
- Pre token generation
- Migrate user
- Custom message
- Custom sender

Example of inserting a user into a
DynamoDB table after creation

```
aws cognito-idp update-user-pool \  
--user-pool-id YOUR_USER_POOL_ID \  
--lambda-config PostConfirmation=YOUR_LAMBDA_FUNCTION_ARN
```

```
import json  
import psycopg2  
  
def lambda_handler(event, context):  
    user = event['request']['userAttributes']  
    try:  
        conn = psycopg2.connect(  
            host=(os.getenv('PG_HOSTNAME')),  
            database=(os.getenv('PG_DATABASE')),  
            user=(os.getenv('PG_USERNAME')),  
            password=(os.getenv('PG_SECRET'))  
        )  
        cur = conn.cursor()  
        cur.execute("""  
INSERT INTO users (display_name, handle, cognito_user_id)  
VALUES(%s, %s, %s)""",  
            (user['name'], user['email'], user['sub']))  
        conn.commit()  
    except (Exception, psycopg2.DatabaseError) as error:  
        print(error)  
    finally:  
        if conn is not None:  
            cur.close()  
            conn.close()  
            print('Database connection closed.')    return event
```

Cognito User Pools – MFA

MFA can be added as part of the Authentication Flow with Amazon Cognito User Pools.

You can have the following settings:

- No MFA
- Optional MFA
 - The user can opt-in to using MFA
- Require MFA
 - All users require MFA to login

MFA can be sent either:

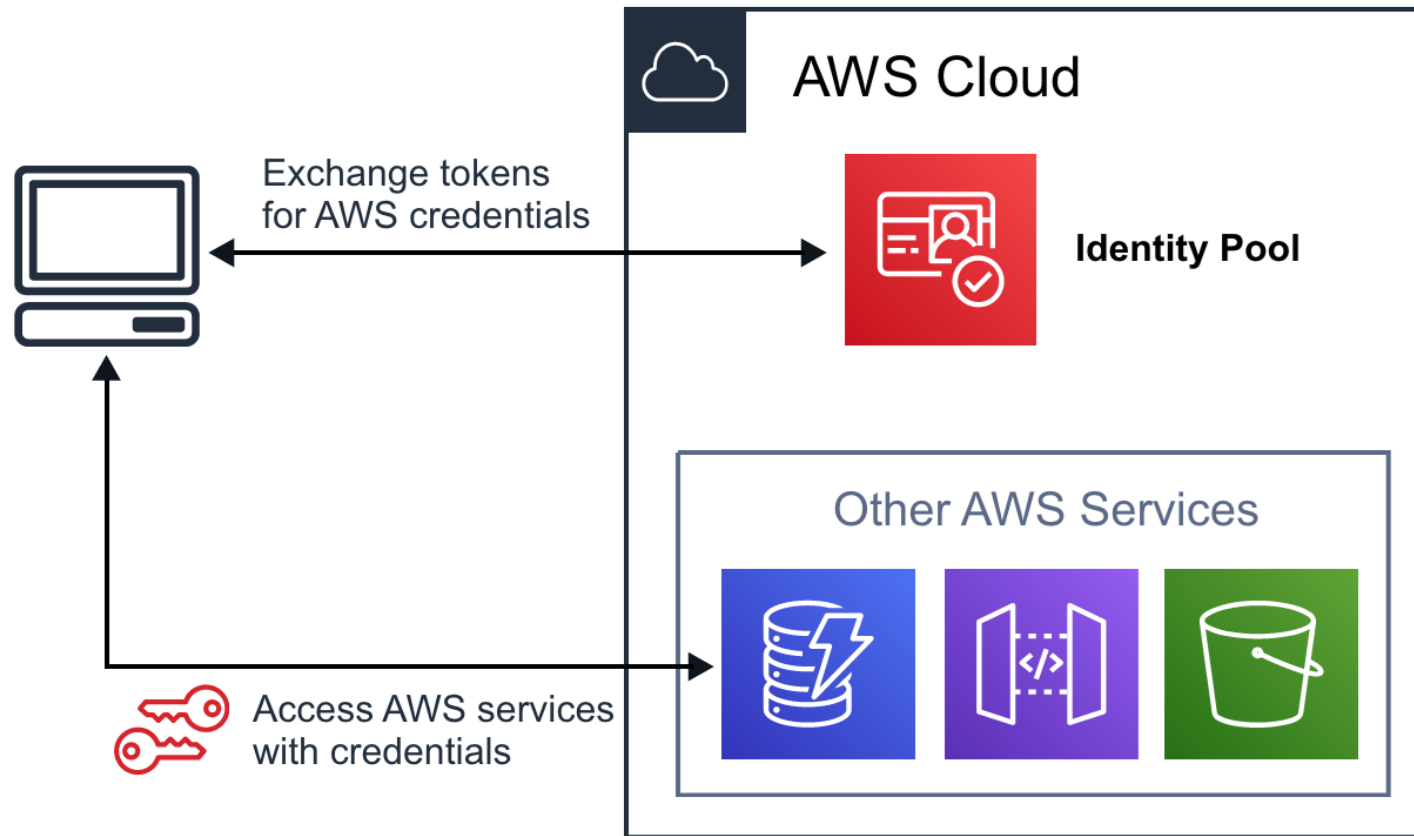
- Text Message eg. SMS
- OTOP software tokens

MFA utilizing SMS will result in additional charges

```
Resources:
  MyCognitoUserPool:
    Type: AWS::Cognito::UserPool
    Properties:
      UserPoolName: MyUserPool
      MfaConfiguration: ON
      SmsAuthenticationMessage: "Your authentication code is {####}."
      SmsConfiguration:
        SnsCallerArn: arn:aws:iam::123456789012:role/sns-calling-role
        ExternalId: "yourExternalIdHere"
      SoftwareTokenMfaConfiguration:
        Enabled: true
      AdminCreateUserConfig:
        AllowAdminCreateUserOnly: false
        UnusedAccountValidityDays: 7
      Schema:
        - AttributeDataType: String
          Name: email
          Required: true
          Mutable: false
```

Cognito Identity Pools

Identity Pools provide **temporary AWS credentials** to access services eg. S3, DynamoDB
Identity Pools can be thought of as the actual mechanism authorizing access to the AWS resources.

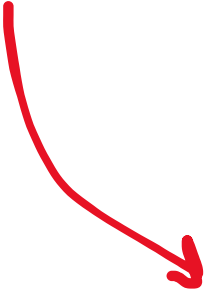


Cognito Identity Pools

You can **provide access to**

- Authentication Identities eg
 - Cognito User Pools
 - Facebook
 - Google
 - Apple
 - Amazon
 - Twitter
 - OIDC
 - SAML
 - Custom developer provider
- Unauthenticated Identities (Guests)

Use the **SDK** to get temporary credentials



```
require 'aws-sdk-cognitoidentity'
require 'aws-sdk-s3'

cognito_identity = Aws::CognitoIdentity::Client.new

identity_pool_id = 'us-east-1:12345678-1...'

resp = cognito_identity.get_id({
  identity_pool_id: identity_pool_id
})

identity_id = resp.identity_id

credentials = cognito_identity.get_credentials_for_identity({
  identity_id: identity_id
}).credentials

s3 = Aws::S3::Client.new(
  credentials: Aws::Credentials.new(
    credentials.access_key_id,
    credentials.secret_key,
    credentials.session_token
  )
)

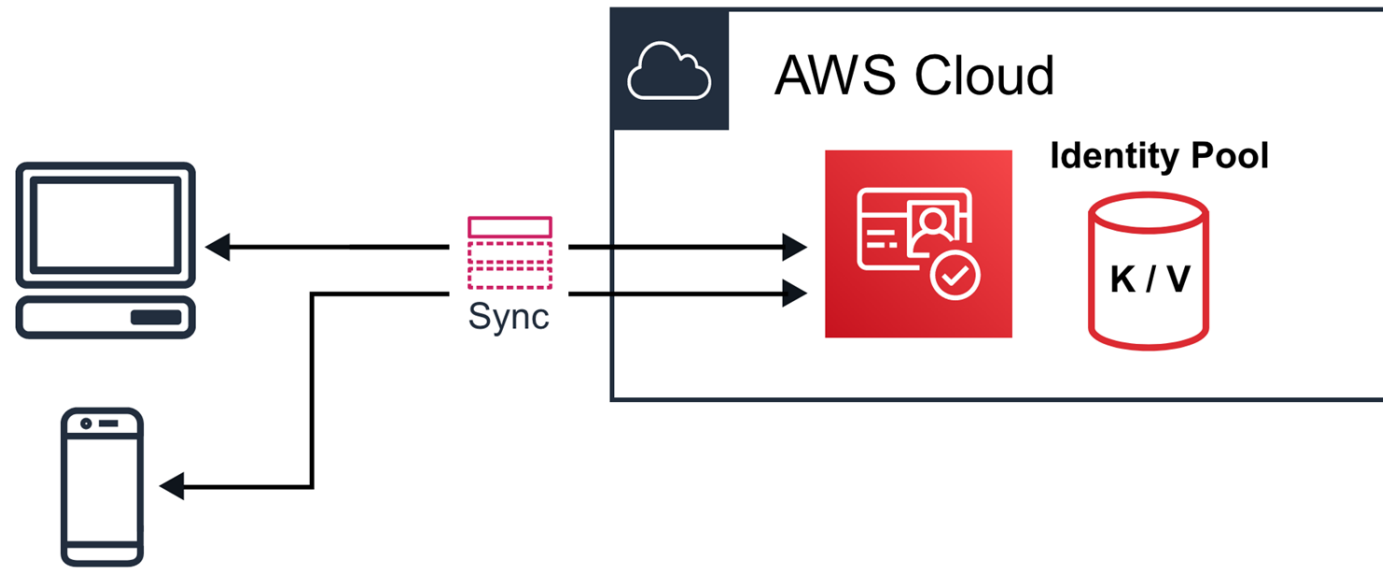
begin
  response = s3.list_buckets
  puts "Buckets:"
  response.buckets.each do |bucket|
    puts bucket.name
  end
rescue Aws::S3::Errors::ServiceError => e
  puts "Failed to list buckets: #{e}"
end
```

Cognito - Sync

Sync **user data** and **preferences** across devices with one line of code
Cognito uses **push synchronization** to push updates and synchronize data



Uses Simple Notification Service (SNS) to send notifications to all user devices when data in the cloud changes.



Cognito - Multi-tenancy

What is Multi-tenancy?

When you have multiple customers (tenants) who use the same application (building) but their **data is logically isolated from each other**. Multi-tenancy is common for SaaS applications where you have organizational-style accounts (multiple users belonging to a group or organization) or you B2B style web-applications

	Advantages	Disadvantages
Per-tenant user pools	• Per-tenant user pools • Extensive customization options	• Higher complexity and cost • Each new tenant increases management overhead
Per-tenant app clients	• Simpler to manage than multiple user pools • Cost-effective	• Limited data isolation • Customization options limited to app client settings
Per-tenant user pool groups	• Simplifies permissions and roles management within a single user pool • Cost-effective and simpler than multiple user pools	• Limited data isolation • Complexity increases with the number of groups and tenants
Per-tenant custom attributes	• Flexible storage of tenant-specific data within user profiles • Simple and cost-effective	• Limited number of custom attributes • Provides less data isolation compared to separate user pools

Cognito – Server-Side Custom Forms

If you want to build your own custom forms you can have the forms submit to your web-application. Your web-app can in-turn call the Amazon Cognito's SDK to handle authentication, authorization endpoints.

```
require 'aws-sdk-cognitoidentityprovider'

client = Aws::CognitoIdentityProvider::Client.new

params = {
  client_id: 'your_app_client_id',
  username: 'example_username',
  password: 'example_password123!',
  user_attributes: [{
    name: 'email',
    value: 'example_email@example.com'
  }]
}

begin
  # Attempt to sign up the user
  response = client.sign_up(params)
  puts "User signed up successfully."
  puts response
rescue Aws::CognitoIdentityProvider::Errors::ServiceError => e
  # If an error occurs, print the error message
  puts "Failed to sign up user: #{e.message}"
end
```

Server Side Advantages

If you don't want to use the Amplify Js library.

If you aren't using JavaScript on client-side

If you not building a web-app, eg mobile app

- *(there is a Mobile App SDK)*

If you want to add additional server-side tracking or actions

Downsides

Increase API calls to your backend app and bottlenecking your authentication service based on the scalability and availability of your backend app

Example of **signup** using AWS Ruby SDK V3

Cognito – Client-Side Custom Forms

To integrate custom form for Amazon Cognito into your web-applications you need to the AWS Amplify Client-side JS SDK

```
npm install aws-amplify --save
```



Amplify UI has react components to quickly implement Authentication forms (Login, Logout, Reset password) however the UX is poorly implemented and will not meet the standard requirements for most apps.



Cognito used to be its own js library but was rolled into amplify js library and so you have to install amplify js.

Cognito – Custom Forms with Amplify

Once you've created your Amazon Cognito User Pool you need to configure Amplify with these settings. You do not need to create your User Pool using Amplify's CLI and it will create more problems if you do.

```
import { Amplify } from 'aws-amplify';
Amplify.configure({
  AWS_PROJECT_REGION: process.env.REACT_APP_AWS_PROJECT_REGION,
  aws_cognito_region: process.env.REACT_APP_AWS_COGNITO_REGION,
  aws_user_pools_id: process.env.REACT_APP_AWS_USER_POOLS_ID,
  aws_user_pools_web_client_id: process.env.REACT_APP_CLIENT_ID,
  oauth: {},
  Auth: {
    // identityPoolId: process.env.REACT_APP_IDENTITY_POOL_ID,
    region: process.env.REACT_APP_AWS_PROJECT_REGION,
    userPoolId: process.env.REACT_APP_AWS_USER_POOLS_ID,
    userPoolWebClientId: process.env.REACT_APP_CLIENT_ID,
  }
});
```



Cognito – Client-Side Custom Forms with Amplify

```
import { Auth } from 'aws-amplify';

export default function SigninPage() {

  const [email, setEmail] = React.useState('');
  const [password, setPassword] = React.useState('');
  const [errors, setErrors] = React.useState([]);

  const onsubmit = async (event) => {
    event.preventDefault();
    setErrors([])
    Auth.signIn(email, password)
      .then(user => {
        console.log('user', user)
        localStorage.setItem(
          "access_token",
          user.signInUserSession.accessToken.jwtToken
        )
        window.location.href = "/"
      })
      .catch(error => {
        if (error.code === 'UserNotConfirmedException') {
          window.location.href = "/confirm"
        }
        setErrors([error.message])
      });
    return false
  }
  // ...
}
```

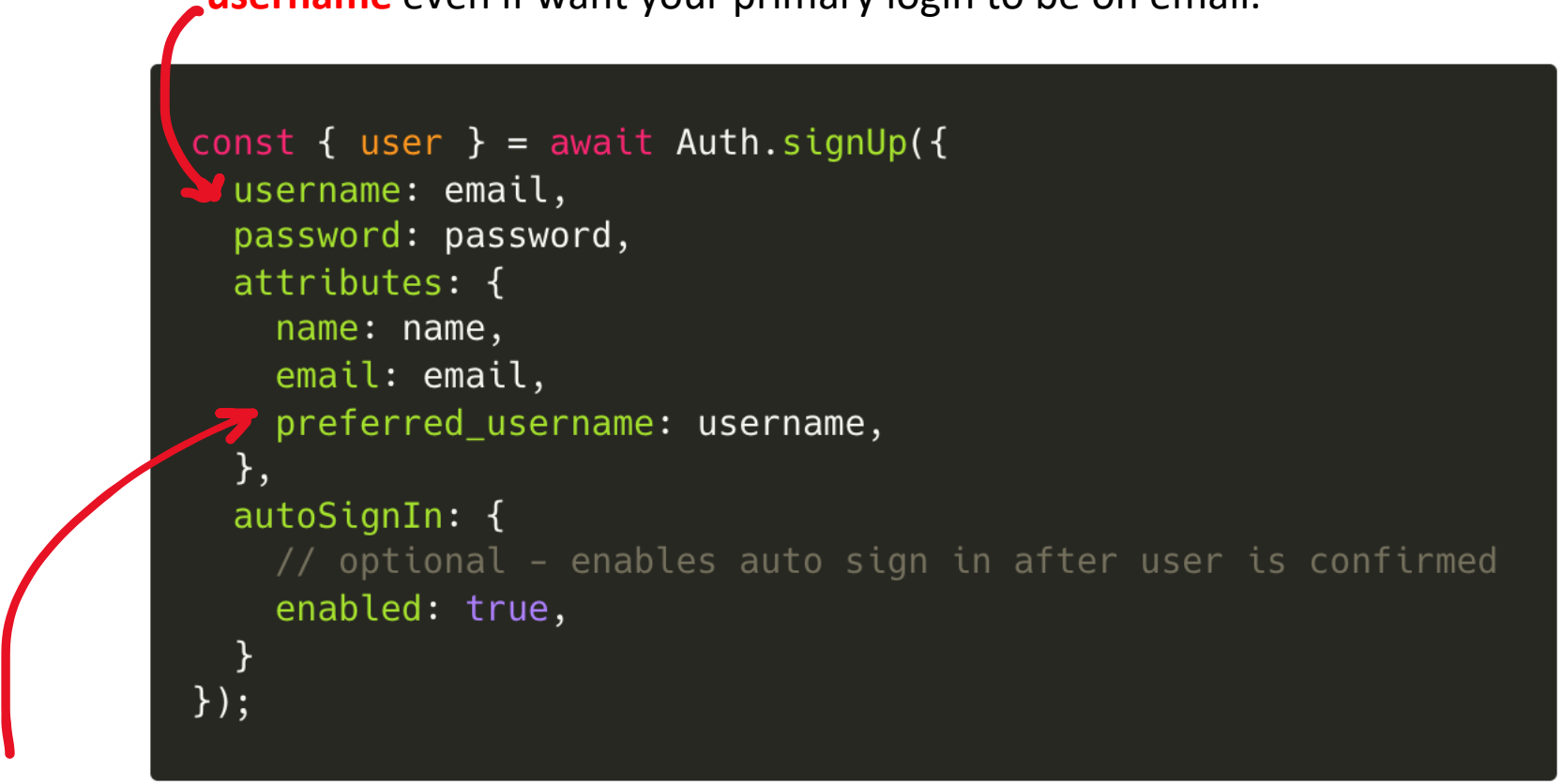
Common API Calls

- **Auth.signIn** — log's a user in creating a new session
- Auth.signUp — creates a new user in the user pool
- Auth.signOut — delete a users sessions
- Auth.getCurrentUser — returns if there is a current session
- Auth.deleteUser — deletes the user for the login pool
- Auth.confirmSignUp — send authorization code
- Auth.resendSignUp — resend confirmation code
- Auth.forgotPassword — send forgot password code
- Auth.forgotPasswordSubmit — reset password

You can store your jwtToken (your session) in **localStorage**

Cognito – Client-Side Custom Forms with Amplify

For signup notice the primary attribute for your identity is called **username** even if you want your primary login to be on email.



```
const { user } = await Auth.signUp({  
  username: email,  
  password: password,  
  attributes: {  
    name: name,  
    email: email,  
    preferred_username: username,  
  },  
  autoSignIn: {  
    // optional - enables auto sign in after user is confirmed  
    enabled: true,  
  }  
});
```

We can still have a username (not for login in this case) we set it to the **preferred_username** field.

Cognito – AWS JWT Verify

AWS JWT Verify is a library by AWS implemented in JavaScript to easily verify JWTs for Cognito.

<https://github.com/aws-labs/aws-jwt-verify>

This library was designed for:

- API Gateway Lambda authorizers
- AppSync Lambda authorizers
- CloudFront Lambda@Edge
- Node.js APIs, e.g. running in AWS Fargate, that need to verify incoming JWTs

```
import { CognitoJwtVerifier } from "aws-jwt-verify";

// Verifier that expects valid access tokens:
const verifier = CognitoJwtVerifier.create({
  userPoolId: "<user_pool_id>",
  tokenUse: "access",
  clientId: "<client_id>",
});

try {
  const payload = await verifier.verify(
    "eyJraWQeyJhdF9oYXNoIjoidk..." // the JWT as string
  );
  console.log("Token is valid. Payload:", payload);
} catch {
  console.log("Token not valid!");
}
```